

K 注意力残差

注意力残差技术报告

Kimi团队

<https://github.com/MoonshotAI/Attention-Residuals>

摘要

现代大语言模型 (LLM) 普遍采用带PreNorm[60]的残差连接[12]，但其以固定单位权重累加所有层的输出。这种均匀聚合方式会导致隐藏状态随网络深度增长而失控膨胀，进而逐步稀释每一层的贡献[27]。我们提出注意力残差 (AttnRes)，以对先前各层输出施加 softmax 注意力的方式替代固定累加机制，使每一层均可依据输入动态学习权重，有选择性地聚合早期表征。为应对大规模模型训练中对全部先前层输出进行注意力计算所引发的内存与通信开销问题，我们引入分块注意力残差 (Block AttnRes)：将网络层划分为若干块，并在块级表征上执行注意力操作，在显著降低内存占用的同时保留绝大部分完整 AttnRes 带来的增益。结合基于缓存的流水线通信机制与两阶段计算策略，Block AttnRes 成为一种实用、即插即用的标准残差连接替代方案，仅带来极小开销。

缩放律实验表明，该改进在不同模型规模下均保持一致；消融实验验证了内容依赖的逐层深度选择所带来的益处。我们进一步将 AttnRes 集成至 Kimi Linear 架构[69]（总参数量 48B / 激活参数量 3B），并在 1.4 万亿词元上进行预训练。在此设置下，AttnRes 缓解了 PreNorm 稀释问题，使各层输出幅值与梯度分布更加均匀，并在所有评估任务上提升了下游性能。

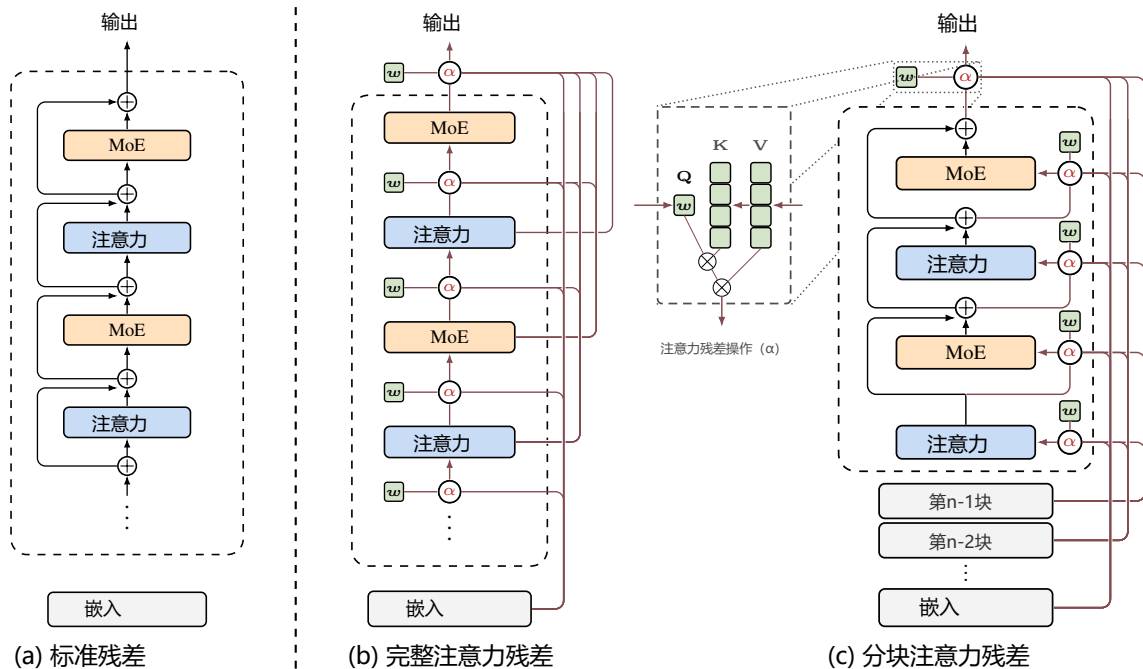


图 1：注意力残差概览。(a) 标准残差：采用均匀加性累加的标准残差连接。(b) 完整 AttnRes：每一层通过可学习的注意力权重选择性聚合所有前序层输出。(c) 分块注意力残差：各层被分组为若干块，内存开销由 $O(Ld)$ 降至 $O(Nd)$ 。

1 引言

标准残差连接 [12] 已成为现代大语言模型 (LLM) [35, 51, 9] 的事实基础构件。更新式 $h_l = h_{l-1} + f_l(h_{l-1})$ 被普遍理解为一类“梯度高速路”，通过恒等映射使梯度绕过变换，从而支持深层网络的稳定训练。然而残差还承担着另一项较少被关注的作用：展开该递推式可见，每一层均接收所有前序层输出的等权重和；残差定义了信息如何在深度方向上聚合。不同于如今已普遍采用可学习、输入依赖加权的序列混合与专家路由机制 [53, 20, 9]，这种深度方向的聚合仍由固定单位权重主导，缺乏机制来选择性地强调或抑制特定层的贡献。

实践中，PreNorm [60] 已成为主流范式，但其无加权累加导致隐状态幅值随深度呈 $O(L)$ 增长，逐步稀释每一层的相对贡献 [27]。早期层的信息被淹没，无法被选择性检索；实证表明，大量层可被剪枝而几乎不损失性能 [11]。近期工作如缩放残差路径 [54] 和多流递推 [72] 仍受限于加性递推结构；而真正引入跨层访问的方法 [36, 56] 则难以规模化。这一困境与注意力机制出现前 RNN 在序列维度上所面临的挑战高度相似。

我们观察到深度方向累加与 RNN 中的时序递推之间存在形式上的对偶性。基于该对偶性，我们提出注意力残差 (AttnRes)，将固定累加 $h_l = \sum_i v_i$ 替换为 $h_l = \sum_i \alpha_i \rightarrow l \cdot v_i$ ，其中 $\alpha_i \rightarrow l$ 是由每层一个可学习伪查询向量 $w_l \in \mathbb{R}^d$ 计算出的 softmax 注意力权重。这一轻量机制仅需每层一个 d 维向量，即可实现深度方向上的选择性、内容感知检索。事实上，标准残差连接及此前基于递推的变体均可被证明执行的是深度方向上的线性注意力；AttnRes 将其推广至深度方向上的 softmax 注意力，从而在深度维度上完成了与序列维度上同样具有变革意义的“线性 $\rightarrow$ softmax”跃迁（第 6.2 节、第 6.1 节）。

在标准训练中，完整版 AttnRes (Full AttnRes) 仅带来可忽略的额外开销，因为其所需的各层输出已在反向传播中被保留。然而在大规模训练场景下，激活重计算与流水线并行被广泛采用，此时这些激活必须被显式保存并在流水线各阶段间通信。为此，我们提出 Block AttnRes 以维持该场景下的效率：将各层划分为 N 个块，每个块通过标准残差压缩为单一表征，跨块注意力仅作用于这 N 个块级摘要。此举将内存与通信开销均降至 $O(Nd)$ ，再结合配套基础设施优化（第 4 节），Block AttnRes 可作为标准残差连接的即插即用替代方案，训练成本极低，推理延迟开销可忽略不计。

缩放律实验确认，AttnRes 在各类计算预算下均持续优于基线模型；其中 Block AttnRes 的性能相当于使用 1.25 倍计算量训练的基线模型。我们进一步将 AttnRes 集成至 Kimi Linear 架构 [69]（总计 48B 参数 / 激活 3B 参数），并在 1.4T tokens 上完成预训练。对所得训练动力学的分析表明：AttnRes 可缓解 PreNorm 稀释现象，使各层输出幅值在深度方向上保持有界，梯度范数在各层间的分布也更趋均匀。在下游基准任务上，最终模型在所有评测任务中均优于基线模型。

主要贡献

- 注意力残差。我们提出 AttnRes，以基于深度的可学习 softmax 注意力取代固定权重的残差累加；并进一步提出其可扩展变体 Block AttnRes，将内存与通信开销从 $O(Ld)$ 降至 $O(Nd)$ 。通过统一的结构化矩阵分析，我们证明：标准残差及此前基于递推的变体均对应于深度方向上的线性注意力，而 AttnRes 则实现深度方向上的 softmax 注意力。
- 面向大规模部署的基础设施。我们开发了系统级优化方案，使分块注意力残差 (Block AttnRes) 在大规模场景下切实可行且高效，包括跨阶段缓存（在流水线并行下消除冗余传输）以及两阶段推理策略（借助在线 softmax [31] 分摊跨块注意力计算）。最终训练开销极小，典型推理负载下的推理延迟开销低于 2%。
- 全面的评估与分析。我们通过缩放律实验、组件消融研究以及下游基准测试，在一个于 1.4T tokens 上预训练的 48B 参数模型上验证了 AttnRes 的有效性，结果表明其在各项指标上持续优于标准残差连接。训练动力学分析进一步揭示：AttnRes 可缓解 PreNorm 稀释现象，使隐状态幅值在各层间保持有界，且梯度范数在深度方向上的分布更为均匀。

2 动机

符号说明。考虑一批输入序列，其形状为 $B \times T \times d$ ，其中 B 为批大小， T 为序列长度， d 为隐层维度。为清晰起见，我们以单个 token 为例写出公式： $h_l \in \mathbb{R}^d$ 表示进入第 l 层的隐状态，其中 $l \in \{1, \dots, L\}$ 为层索引， L 为总层数；token 嵌入为 h_1 ；函数 f_l 表示第 l 层所执行的变换。在 Transformer 模型中，我们将每个自注意力模块或 MLP 视为一个独立的层。

2.1 通过残差训练深层网络

残差学习。残差学习 [12] 已被证明是训练深层网络的一项关键技术，因其允许梯度绕过变换过程。具体而言，每一层按如下方式更新隐状态：

$$h_l = h_{l-1} + f_{l-1}(h_{l-1})$$

展开该递推式可知，第 l 层的隐状态等于嵌入向量与所有前序层输出之和：

$h_l = h_1 + \sum_{i=1}^{l-1} f_i(h_i)$ 。残差连接背后的核心洞见在于恒等映射：每一层均保留一条直达通路，信息与梯度均可无损流通的通路。在反向传播过程中，中间隐状态关于损失的梯度为：

$$\frac{\partial \mathcal{L}}{\partial h_l} = \frac{\partial \mathcal{L}}{\partial h_L} \cdot \prod_{j=l}^{L-1} \left(\mathbf{I} + \frac{\partial f_j}{\partial h_j} \right)$$

展开该乘积后得到单位矩阵 $\mathbf{I}$ 加上涉及各层雅可比矩阵 $\partial f_j / \partial h_j$ 的高阶项。单位项始终被保留，从而为损失函数到任意层（无论深度如何）提供了直接的梯度通路。

残差的泛化。尽管有效，但残差更新中固定的单位系数对每一层的贡献一视同仁，无法在不同深度间自适应地调整混合权重。高速公路网络 (Highway networks) [45] 通过引入可学习的逐元素门控机制来放宽这一限制：

$$h_l = (1 - g_l) \odot h_{l-1} + g_l \odot f_{l-1}(h_{l-1})$$

其中 $g_l \in [0, 1]^d$ 在变换路径与恒等路径之间进行插值。更一般地，二者均为加权递归 $h_l = \alpha_l \cdot h_{l-1} + \beta_l \cdot f_{l-1}(h_{l-1})$ 的实例，残差设置为 $\alpha_l = \beta_l = 1$ ，Highway 设置为 $\alpha_l = 1 - g_l$ ， $\beta_l = g_l$ 。

局限性。无论是固定型还是门控型，这两种方法均共享一个根本约束：每一层只能访问其直接输入 h_{l-1} ——即一个融合了所有先前层输出的单一压缩状态，而非各层输出本身。这带来若干局限：（1）缺乏选择性访问：不同类型层（如注意力层与 MLP 层）接收相同的聚合状态，尽管它们可能从不同权重中获益；（2）不可逆的信息损失：经聚合丢失的信息无法在更深层中被选择性恢复；（3）输出增长：后层需学习越来越大的输出以增强其在累积残差中的影响，这可能导致训练不稳定。这些局限性促使我们设计一种机制，使每一层能选择性地聚合所有先前层的信息。

3 注意力残差：时间与深度的统一视角

上述局限性令人联想到序列建模中类似的瓶颈，提示我们应为深度维度寻求相似的解决方案。

时间与深度的对偶性。如同 RNN 对时间建模，残差连接将所有先前信息沿深度压缩为单一状态 h_l 。在序列建模中，Transformer 通过以注意力替代循环结构 [3, 52] 改进了 RNN，使每个位置能够依据数据依赖的权重选择性地访问所有先前位置。我们为深度维度提出相同的方法论：

$$h_l = \alpha_{0 \rightarrow l} \cdot h_1 + \sum_{i=1}^{l-1} \alpha_{i \rightarrow l} \cdot f_i(h_i) \quad (1)$$

其中 $\alpha_{i \rightarrow l}$ 是满足 $\sum_{i=0}^{l-1} \alpha_{i \rightarrow l} = 1$ 的层特定注意力权重， $i=0$ 时 $\alpha_{0 \rightarrow l} = 1$ 。不同于序列长度（可能达到百万级词元），网络深度通常较为适中（ $L < 1000$ ），使得对深度的 $O(L^2)$ 注意力在计算上可行。我们将该方法称为注意力残差 (Attention Residuals)，简称为 AttnRes。

3.1 全注意力残差

注意力权重可写作 $\alpha_{i \rightarrow l} = \phi(q_l, k_i)$, 其中 $\phi: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$ 是一个核函数, q_l 和 k_i 分别为查询与键向量 [23, 70]。 ϕ 的不同选择可复现不同的残差变体 (§6.2) ; 我们采用 $\phi(q, k) = \exp q^\top \text{RMSNorm}(k)$ [66] 并施加归一化, 从而得到对深度的 softmax 注意力:

$$\alpha_{i \rightarrow l} = \frac{\phi(q_l, k_i)}{\sum_{j=0}^{l-1} \phi(q_l, k_j)} \quad (2)$$

对每一层 l , 我们定义:

$$q_l = w_l, \quad k_i = v_i = \begin{cases} h_1 & i = 0 \\ f_i(h_i) & 1 \leq i \leq l-1 \end{cases} \quad (3)$$

其中查询 $q_l = w_l$ 是 $\mathbb{R}^d$ 中一个层特定的可学习向量。 ϕ 内部的 RMSNorm 可防止输出幅值较大的层主导注意力权重。第 l 层的输入则为:

$$h_l = \sum_{i=0}^{l-1} \alpha_{i \rightarrow l} \cdot v_i \quad (4)$$

我们将此形式称为全注意力残差。对每个词元, Full AttnRes 需要 $O(L^2d)$ 的算术运算量以及 $O(Ld)$ 的内存来存储层输出。由于网络深度远小于序列长度, 其算术开销相对适中。

开销。 $O(Ld)$ 内存开销完全与反向传播所需保留的激活值重叠, 因此 Full AttnRes 在标准训练中不引入额外内存开销。但在大规模场景下, 激活重计算与流水线并行被广泛采用: 原本可被释放并重计算的层输出现在必须为所有后续层持续保留; 而在流水线并行下, 每一层输出还需进一步跨阶段边界传输。此时, 内存与通信开销均以 $O(Ld)$ 增长。

分块优化。 Full AttnRes 中一项刻意的设计选择是伪查询 w_l 为一个与各层前向计算解耦的学习参数。这种独立性意味着任意一组层的注意力权重均可并行计算, 而无需等待其顺序输出; 尤其允许将 L 层分组为每组 S 层的 N 个块, 并在每个块内批量执行注意力计算, 从而将每层内存 I/O 开销从 $O(Ld)$ 降至 $O((S+N)d)$ (详细两阶段策略见 §4)。然而, 在当前分布式训练范式下, 主导成本并非本地内存带宽, 而是流水线并行下的跨阶段通信: 每一层输出仍需在各阶段间传输, 而这一 $O(Ld)$ 通信开销无法通过本地批处理缓解。这促使我们引入下方所述的 Block AttnRes 变体, 将跨阶段表示的数量从 L 减少至 N 。我们预期未来互连技术的改进将使完整的 $O(Ld)$ 通信成为现实, 从而充分释放 Full AttnRes 的潜力。

3.2 块注意力残差

我们提出块注意力残差 (Block Attention Residuals), 将 L 层划分为 N 个块: 在每个块内部, 各层输出通过求和缩减为单一表示; 在块之间, 则仅对 N 个块级表示及词元嵌入应用全注意力。这将内存与通信开销从 $O(Ld)$ 降低至 $O(Nd)$ 。

块内累加。具体而言, 我们将 L 层划分为 N 个块, 每块包含 $S = L/N$ 层 (假设 L 可被 N 整除; 否则, 最后一块包含剩余的 $L \bmod N$ 层)。令 B_n 表示第 n 个块 ($n = 1, \dots, N$) 中的层索引集合。为构建一个块, 我们将该块中所有层的输出相加:

$$b_n = \sum_{j \in B_n} f_j(h_j) \quad (5)$$

我们进一步将 b_n 记作 B_n 中前 i 层的部分和, 从而有 $b_n = b_{S_n}$ 。当 L 不能被 N 整除时, 最终的部分和取为最后一个块的表示。与 Full AttnRes 一样, ϕ 内部的 RMSNorm 可防止完整块与部分和之间在幅值上的差异对注意力权重造成偏差。

```

1 def block_attn_res(blocks: list[Tensor], partial_block: Tensor, proj: Linear, norm: RMSNorm) -> Tensor:
2     """
3     块间注意力：对块表示与部分和进行注意力计算。
4     blocks:
5         N 个形状为 [B, T, D] 的张量：每个先前块的已完成块表示
6     partial_block:
7         [B, T, D]：块内部分和 (b_n^i)
8     """
9     V = torch.stack(blocks + [partial_block]) # [N+1, B, T, D]
10    K = norm(V)
11    logits = torch.einsum('d, n b t d -> n b t', proj.weight.squeeze(), K)
12    h = torch.einsum('n b t, n b t d -> b t d', logits.softmax(0), V)
13    return h
14
15 def forward(self, blocks: list[Tensor], hidden_states: Tensor) -> tuple[list[Tensor], Tensor]:
16     partial_block = hidden_states
17     # 在注意力前应用块注意力残差
18     # blocks已包含词元嵌入
19     h = block_attn_res(blocks, partial_block, self.attn_res_proj, self.attn_res_norm)
20
21     # 若到达块边界，则启动新块
22     # block size计为ATTN + MLP；每个Transformer层含2个
23     if self.layer_number % (self.block_size // 2) == 0:
24         blocks.append(partial_block)
25         partial_block = None
26
27     # 自注意力层
28     attn_out = self.attn(self.attn_norm(h))
29     partial_block = partial_block + attn_out if partial_block is not None else attn_out
30
31     # 在MLP前应用块注意力残差
32     h = block_attn_res(blocks, partial_block, self.mlp_res_proj, self.mlp_res_norm)
33
34     # MLP层
35     mlp_out = self.mlp(self.mlp_norm(h))
36     partial_block = partial_block + mlp_out
37
38     return blocks, partial_block

```

图2：Block Attention Residuals的PyTorch风格伪代码。block_attn_res利用可学习伪查询 w_l ，在块表示上执行softmax注意力；forward为单层前向过程，维护partial_block (b_n ，块内残差) 与blocks ($[b, \dots, b]$ ，块间历史)。

块间注意力。在完整AttnRes中，第 i 层输入通过对至 $i-1$ (h_{i-1}) 为止的所有输出执行注意力得到。块式变体将这些独立输出替换为块表示，并定义 $b = h$ ，以确保词元嵌入始终作为源参与。对于块 n 中的第 i 层，值矩阵为：

$$V = \begin{cases} [b, b, \dots, b]^T & \text{当 } i = 1 \text{ 时 (块 } n \text{ 的首层)} \\ [b, b, \dots, b, b_{i-1n}]^T & (\text{当 } i \geq 2 \text{ 时, 即块 } n \text{ 中的后续层)} \end{cases} \quad (6)$$

键与注意力权重分别遵循公式3与公式2。网络首层输入为词元嵌入，即 $b = h$ 。在每个块中，首层接收前序块表示与词元嵌入，后续层额外对部分和 b_{i-1n} 执行注意力。最终输出层聚合全部 N 个块表示。图2提供了Block AttnRes的PyTorch风格伪代码。

效率。由于每层现仅需对 N 个块表示执行注意力计算，而非对 L 个独立输出进行计算，内存占用从 $O(L)$ 降至 $O(N)$ ，计算量从 $O(L^2)$ 降至 $O(N^2)$ 。块数 N 介于两个极端之间：当 $N = L$ 时，退化为完整的AttnRes；当 $N = 1$ 时，则简化为标准残差连接，此时嵌入被单独隔离为 b 。实验表明， $N \approx 8$ 可在不同模型规模下恢复大部分收益，每token仅需存储8个隐藏状态（见第5节）。

除内存与计算外，块结构亦有助于降低推理延迟：块边界定义了第3节所述分块优化的调度粒度，且固定块数 N 限定了KV缓存大小。块间并行结果通过在线softmax[31]与块内顺序部分和相融合，保持严格等价性（见第4节）。

4 基础设施设计

Block AttnRes相较于标准残差连接带来了额外的系统挑战。在大规模模型训练中，块表示必须在流水线各阶段间传递，从而导致高通信负载。

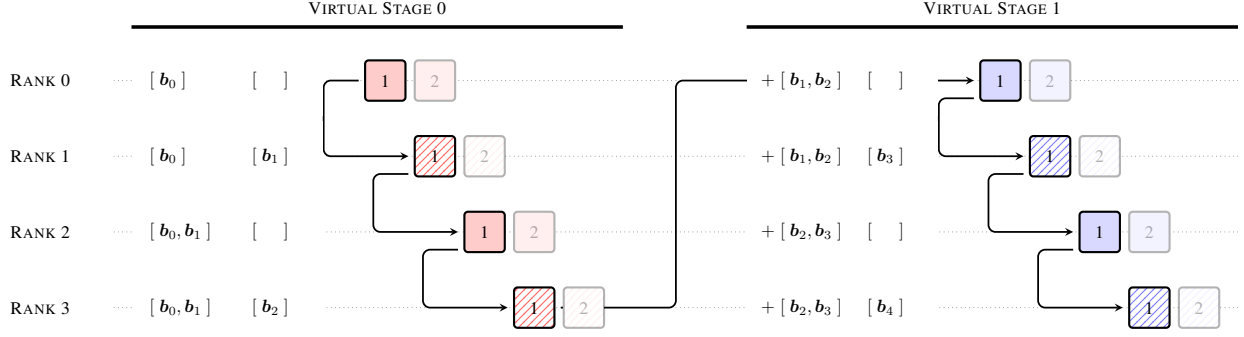


图3: 基于缓存的流水线通信示例, 共4个物理rank, 每个rank包含2个虚拟阶段, 其中带斜线阴影的方框表示AttnRes块的结束位置。数字表示微批次索引。每个rank缓存此前接收到的块; 阶段切换仅传输增量块 (+[b1, b2]), 而非完整历史。

朴素实现。在推理阶段, 重复访问累积的块表示会增加延迟, 而长上下文预填充则会放大缓存块表示的内存开销。我们通过训练阶段的跨阶段缓存, 以及推理阶段的两阶段计算策略与内存高效的预填充方案来应对这些挑战。

4.1 训练

对于小规模训练, AttnRes 仅引入极小的计算开销且不增加额外内存占用, 因为激活值无论如何均需保存以支持反向传播。在大规模分布式训练中, 流水线并行对 AttnRes 构成主要基础设施挑战。完整版 AttnRes 需跨阶段传输全部 L 层输出; Block AttnRes 将其缩减为 N 个块表示, 而下述优化进一步最小化剩余开销。

流水线通信。采用标准残差连接时, 流水线并行 [18] 在相邻阶段间传输固定尺寸的隐藏状态, 与流水线深度无关。Block AttnRes 要求各阶段具备全部已累积的块表示以执行块间注意力, 而若在每次转换时朴素地传输完整历史记录, 则会引发冗余通信。

考虑一种交错流水线调度 [33], 其包含 P 个物理阶段, 每个物理阶段含 V 个虚拟阶段。为简化起见, 假设每个物理阶段平均每 Token 生成 N_p 个维度为 d 的块表示。¹ 设 $C = PV$ 为总分块数 (即每个虚拟阶段中的每个物理阶段), 则第 j 个分块累积 jN_p 个块。若在每次转换时朴素地传输所有已累积块, 则每 Token 的通信开销为:

$$\text{Comm}_{\text{朴素}} = \sum_{j=1}^{C-1} jN_p \cdot d = \frac{C(C-1)}{2} N_p d. \quad (7)$$

跨阶段缓存。由于每个物理阶段依次处理多个虚拟阶段, 我们可通过本地缓存块来消除该冗余: 在较早虚拟阶段接收的块保留在本地内存中, 无需重新传输。第一个虚拟阶段 ($v = 1$) 无缓存, 按常规累积; 对于 $v \geq 2$, 每次转换仅传递自前一虚拟阶段对应接收方分块以来累积的约 PN_p 个增量块。总通信量缩减为:

$$\text{Comm}_{\text{cached}} = \underbrace{\frac{P(P-1)}{2} N_p d}_{|z| \text{ 第一个虚拟阶段}} + \underbrace{(V-1) P^2 N_p d}_{|z| \text{ 后续虚拟阶段}}. \quad (8)$$

缓存将每次转换的峰值开销从 $O(C)$ 降至 $O(P)$, 实现了 V 倍的提升, 从而在稳态 1F1B 期间实现与计算的完全重叠。反向传播同样受益于该方案。图 3 以 $P=4$ 和 $V=2$ 展示了该优化: 对于第二个虚拟阶段, 缓存消除了 6 次冗余的块传输。

¹ 实际中, 块边界无需与物理阶段边界对齐。例如, 在图 3 中, 每个块跨越两个物理阶段, 因此仅每隔一次转换涉及一个新完成的块。

算法 1: 块 n 的两阶段计算

```

    输入: 伪查询  $\{w_l\}_{l \in B_n}$ , 块表示  $\{b_0, \dots, b_{n-1}\}$ 
    /* 第一阶段: 块间并行注意力
1    $Q \leftarrow [w_l]_{l \in B_n}$                                      //  $[S, d]$ 
2    $K, V \leftarrow [b_0; \dots; b_{n-1}]$                          //  $[n, d]$ 
3    $\{o_l^{(1)}, m_l^{(1)}, \ell_l^{(1)}\}_{l \in B_n} \leftarrow \text{ATTNWITHSTATS}(Q, K, V)$  // 返回 LSE
4
    /* 第二阶段: 块内顺序注意力 + 在线 Softmax 合并
5    $i \leftarrow 0$ 
6   对  $l \in B_n$  执行循环
7   若  $i = 0$ , 则
8        $h_l \leftarrow o_l^{(1)} / \ell_l^{(1)}$                         // 仅块间
9   否则
10       $o_l^{(2)}, m_l^{(2)}, \ell_l^{(2)} \leftarrow \text{ATTNWITHSTATS}(w, b_l^n, b_l^n)$  // 块内
11       $m_l \leftarrow \max(m_l^{(1)}, m_l^{(2)})$ 
12   $h \leftarrow \frac{e^{(1)} - m_l^{(1)} + e^{(2)} - m_l^{(2)}}{e^{(1)} - m_l^{(1)} + e^{(2)} - m_l^{(2)}}$  // 在线 softmax 合并
13       $i \leftarrow i + 1$ 
14   $b_l^n \leftarrow b_l^n + f(h)$  // 更新局部和;  $b^n := 0$ 
15 返回  $\{h_l\}_{l \in B_n}$ 

```

内存开销。借助跨阶段缓存, 每个块在整个 V 个虚拟阶段中仅被存储一次, 其开销相对于标准的每层激活缓存已可忽略。关键在于, 每层激活占用仍与标准架构完全一致: 激活检查点机制消除了所有块间注意力中间结果, 而被检查点化的输入 pl 在内存大小上与其所替代的隐藏状态 hl 完全相同。

就实际运行时间而言, 当未启用流水线并行时, Block AttnRes 带来的训练开销可忽略不计; 启用流水线并行后, 实测端到端开销低于 4%。

4.2 推理

以下所述的两阶段计算策略适用于 Full 和 Block AttnRes 两种情形: 无论哪种情形, 层均被分组为大小为 S 的块, 第一阶段批量处理块间查询, 第二阶段顺序处理块内回溯。对于 Full AttnRes, 此策略将每层 I/O 从 $O(Ld)$ 降至 $O((S+N)d)$ (详细推导见附录 B); Block AttnRes 进一步将存储的表示数量由 L 减少至 N , 因为每个块被压缩为单一向量。后续内容将聚焦于 Block AttnRes, 并详述该两阶段计算策略, 以及面向长上下文输入的序列分片预填充方案。

两阶段计算策略。Block AttnRes 的逐层注意力计算类似于自回归解码, 其中块表示充当跨层共享的 KV 缓存。朴素实现会在每一层都计算注意力残差, 每次均需遍历所有前置块, 导致 $O(L \cdot N)$ 级别的内存访问量。由于伪查询向量与前向计算解耦 (见 §3), 块内全部 $S = L/N$ 个查询可批处理为单次矩阵乘法, 从而将内存访问从 S 次摊销至 1 次。算法 1 实例化了这一利用上述特性的两阶段计算策略。

- 第一阶段通过单次批处理查询, 对全部 S 层同步执行块间注意力计算, 查询目标为缓存的块表示, 并同时返回输出及 softmax 统计量 (最大值与对数和指数)。此举摊销了内存访问开销, 将读取次数由 S 次降至每块仅 1 次。
- 第二阶段利用不断演进的局部和, 依次为块内每一层顺序计算层内注意力, 再通过在线 softmax [31] 与第一阶段输出合并。由于在线 softmax 合并是逐元素操作, 该阶段天然支持与周边运算的核融合, 从而进一步降低 I/O 开销。

借助两阶段设计, 第二阶段保留了与标准残差连接相近的 I/O 占用, 而主要额外开销则来自第一阶段的块间注意力计算。由于这些块间读取操作可在多个层之间摊销,

通过将块内所有层进行批处理，每层总内存访问开销仅为 $(NS + 3)d$ 次读取和 $2d$ 次写入（见表 1）。该开销显著低于先前各类残差泛化方法（例如 $(m)HC$ ）在典型设置下的残差流 I/O 开销。实践中，第一阶段还可与块内首层的计算部分重叠，进一步降低其对实际运行时间的影响。因此，在典型推理负载下，端到端推理延迟开销低于 2%。

表 1：各方案下残差机制每层每 Token 所产生的内存访问开销。层函数 fl 的内部 I/O 不计入其中。对于 AttnRes, Full 和 Block 两种变体均采用附录 B 中所述的两阶段推理调度；摊销开销按块内 N 层取平均值。典型参数取值： $L=128$, $N=8$, $S=L/N=16$, $m=4$ 。

操作		读取	写入	总 I/O	
				符号化	典型
标准残差 残差合并		$2d$	d	$3d$	$3d$
mHC (m 个流)	计算 αl 、 βl 、 Al		m^2+2m		
	应用 αl	$md+m$	d		
	应用 βl	$d+m$	md	$(8m+2)d+2m^2+4m$	$34d$
	应用 Al	$md+m^2$	md		
	残差合并	$2md$	md		
AttnRes	全	第一阶段（摊销） $(N-1)d$ 24d 第二阶段 $(S-1)d$ d		$(S+N)d$	
	分块	第一阶段（摊销） $5.5d$ 第二阶段 $3d$ d	$\frac{N}{S}d$ d	$(\frac{N}{S}+5)d$	

内存高效的预填充。预填充期间存储块表示需占用 $N \cdot T \cdot d$ 个元素，在 128K Token 序列且含 8 个块的情况下将消耗 15 GB 内存。我们通过沿序列维度将这些表示分片至 P 个张量并行设备上缓解此问题，使得第一阶段可在本地序列分片上独立执行。第二阶段在线 softmax 合并则自然融入标准 TP all-reduce 通信路径：输出先经规约分发（reduce-scatter），再于本地合并，最后通过全聚集（all-gather）重建，从而天然支持与 RMSNorm 等操作的内核融合。此举将单设备内存占用降至 $N \cdot (T/P) \cdot d$ ——以 128K 上下文为例，单设备内存从 15 GB 降至约 1.9 GB。结合分块预填充（如 16K 分块大小），单设备开销进一步降低至 0.3 GB 以内。

5 实验

架构细节。我们的架构与 Kimi Linear [69] 完全一致，即一种遵循 Moonlight [28] / DeepSeek-V3 [9] 设计的专家混合（MoE）Transformer，其以 3:1 的比例交替堆叠 Kimi Delta 注意力（KDA）层与多头潜在注意力（MLA）层，且每一层后均接一个 MoE 前馈层。唯一改动是在残差连接中引入 AttnRes；其余所有组件（模型深度、隐藏维度、专家路由及 MLP 结构）均保持不变。AttnRes 仅在每层引入一个 RMSNorm 和一个伪查询向量 $w_l \in \mathbb{R}^d$ ，其参数总量仅占模型总参数量的可忽略部分。关键在于，所有伪查询向量必须初始化为零。这确保了初始注意力权重 $\alpha_i \rightarrow l$ 在各源层间均匀分布，使 AttnRes 在训练起始阶段退化为等权平均，从而避免训练波动——我们已通过实验验证了这一点。

5.1 缩放律

我们在五个模型规模（见表 2）上进行遍历，并对每个规模训练三种变体：PreNorm 基线、全注意力残差以及含约 8 个块的分块注意力残差。它们均采用 8192 Token 的上下文窗口和余弦学习率调度进行训练。在每组缩放律规模内，所有变体共享基线所选定的相同超参数，以确保公平比较；该设置有意偏向基线，从而使比较更为保守。依照标准实践，我们拟合形式为 $L = A \times C^{-\alpha}$ [22, 15] 的幂律曲线，其中 L 为验证损失， C 为以 PFLOP/秒-天为单位测得的计算量。

缩放行为。图 4 展示了拟合的缩放曲线。基线遵循 $L = 1.891 \times C^{-0.057}$ ，分块注意力残差拟合为 $L = 1.870 \times C^{-0.058}$ ，全注意力残差拟合为 $L = 1.865 \times C^{-0.057}$ 。这三种变体均展现出相似的斜率，但注意力残差在整个计算量范围内始终实现更低的损失。根据拟合曲线，在 5.6

表 2: 基线 vs 分块注意力残差 (N = 8) vs 全注意力残差 vs mHC(-lite) [64]: 模型配置、超参数及验证损失。

激活数							批大小‡	验证损失			
参数†	Token 数	Lb	H	dmodel	dff	***		基线	分块注意力残差	全注意力残差	mHC(-lite)
194M	38.7B	12	12	896	400	2.99×10^{-3}	192	1.931	1.909	1.899	1.906
241M	45.4B	13	13	960	432	2.80×10^{-3}	256	1.895	1.875	1.874	1.869
296M	62.1B	14	14	1024	464	2.50×10^{-3}	320	1.829	1.809	1.804	1.807
436M	87.9B	16	16	1168	528	2.20×10^{-3}	384	1.766	1.746	1.737	1.747
528M	119.0B	17	17	1264	560	2.02×10^{-3}	432	1.719	1.693	1.692	1.694

†表示我们 MoE 模型中激活参数的数量 (不含词嵌入)。‡所有模型均以 8192 的上下文长度进行训练。

*Lb = L/2 表示 Transformer 块的数量。

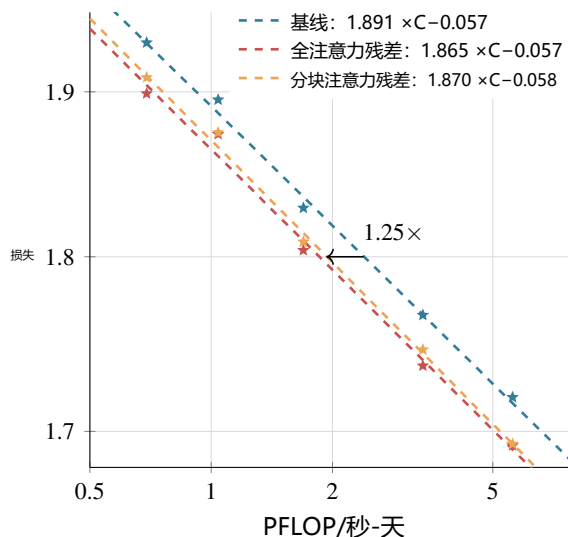


图4: 注意力残差的缩放律曲线。Full与Block AttnRes在所有规模下均持续优于基线模型。Block AttnRes与Full AttnRes高度接近, 在最大规模下几乎完全复现其增益。

PFLOP/s-天: Block AttnRes为1.692, 基线模型为1.714, 相当于获得1.25倍的计算效率优势。Full与Block AttnRes之间的差距随模型规模增大而收窄, 在最大规模下仅相差0.001。表2中亦列出mHC(-lite) [64] 作为参考。Full AttnRes性能优于mHC, 而Block AttnRes在更低的每层内存I/O (5.5d vs. mHC在m=4流下的34d, 见表1) 条件下达到与其相当的性能。

5.2 主要结果

训练配置。本研究最大的模型基于完整的Kimi Linear 48B架构: 共27个Transformer块 (54层), 其中256个专家中有8个被路由激活, 并额外包含1个共享专家, 总参数量为48B, 每步激活参数量为3B。该模型采用Block AttnRes结构, 每块含6层, 共构成9个块, 加上词元嵌入层, 总计10个深度方向的信息源。

我们沿用Kimi Linear 1.4T-token训练流程 [69] 中的数据与训练配置: 所有模型均采用4096-token上下文窗口、Muon优化器 [28] 及WSD (预热-稳定-衰减) 学习率调度策略 [16], 全局批量大小为8M tokens。最终模型的训练分为两个阶段: (i) 在1T tokens上执行WSD预训练; (ii) 随后在约400B高质量tokens上执行中期训练, 遵循Moonlight [28] 的退火策略。

中期训练后, 我们继续以逐步延长的序列长度 (32K tokens) 进行训练。由于本架构采用混合KDA/MLA注意力机制 [69], 而MLA在无位置编码 (NoPE) [61] 条件下运行, 因此上下文扩展无需YaRN [37] 或注意力温度重标定等额外修改。

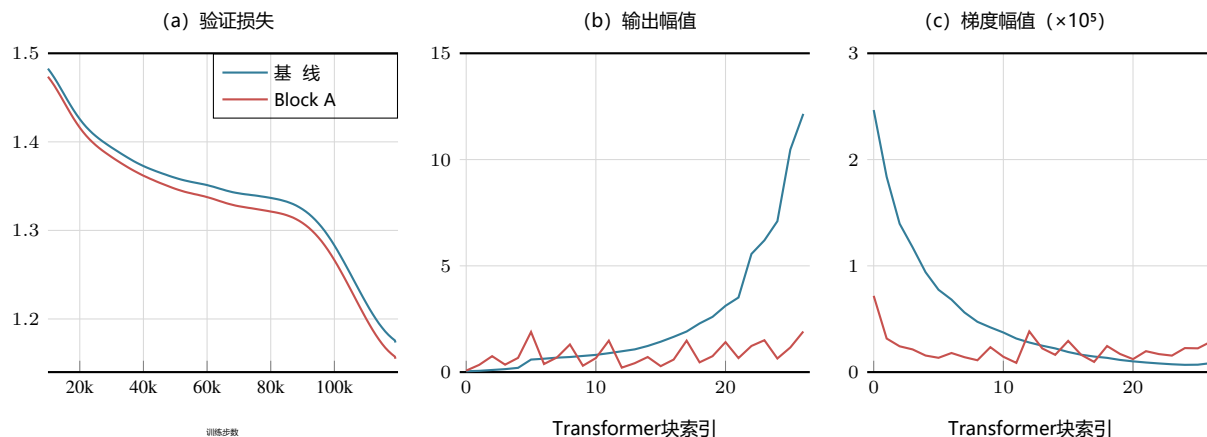


图5：基线模型与Block AttnRes的训练动态。(a) 训练过程中的验证损失；(b) 训练结束时各Transformer块的输出幅值；(c) 各Transformer块的梯度幅值。

训练动态。我们在图5中对比了最终基线模型与Block AttnRes模型在1T token上的训练动态。

- 验证损失：AttnRes在整个训练过程中始终取得更低的验证损失；该差距在衰减阶段进一步扩大，最终损失显著降低。
- 输出幅值：基线模型受PreNorm稀释问题 [60, 27] 困扰：随着隐藏状态幅值随网络深度单调增长，深层模块被迫从固定尺度的归一化输入中学习越来越大的输出，以维持其影响力。Block AttnRes将这种增长限制在每个块内部，块边界处的选择性聚合重置了累积过程，从而形成有界的周期性模式。
- 梯度幅值：当所有残差权重固定为1时，基线模型无法调节跨深度的梯度流，导致最浅层梯度异常增大。Block AttnRes中的可学习softmax权重（图8）在各信息源之间引入概率质量的竞争，从而显著改善梯度分布的均匀性。

表3：AttnRes与基线模型在相同预训练流程后的性能对比。每行最优结果加粗显示。

		基线AttnRes	
通用	MMLU	73.5	74.6
	MMLU-Pro	52.2	52.2
	GPQA-Diamond	36.9	44.4
	BBH	76.3	78.0
	ARC-Challenge	64.6	65.7
	HellaSwag	83.2	83.4
	TriviaQA	69.9	71.8
数学与代码	GSM8K	81.7	82.4
	MGSM	64.9	66.1
	数学		57.1
	CMath	84.7	85.1
	HumanEval	59.1	62.2
	MBPP	72.0	73.9
中文	CMMLU	82.0	82.9
	C-Eval	79.6	82.5

下游性能。我们遵循Kimi Linear [69] 的评估协议，在三个领域对两种模型进行评估（见表3）：

表4: AttnRes关键组件的消融实验 (16层模型)。

变体	损失
基线模型 (PreNorm)	
DenseFormer [36]	1.767
mHC [59]	1.747
AttnRes 完整	
含输入相关查询	1.731
含输入无关混合 1.749 含 sigmoid	1.741
无 RMSNorm	1.743
SWA ($W = 1 + 8$)	
块式 ($S = 4$)	
含多头 ($H = 16$)	1.752
无 RMSNorm	1.750

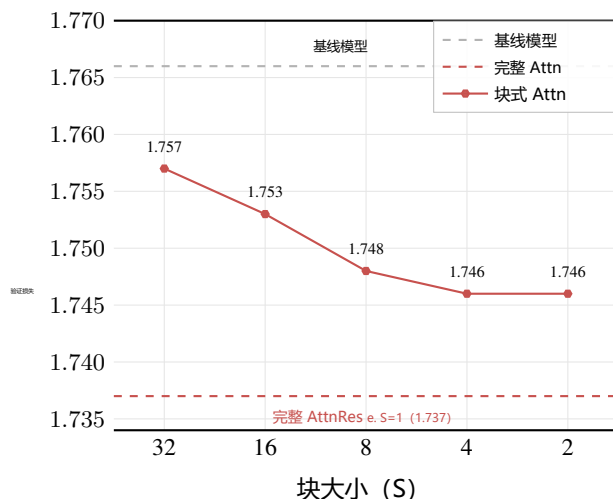


图 6: 块大小对验证损失的影响 (16 层模型)。

- 语言理解与推理: MMLU [13]、MMLU-Pro Hard [55]、GPQA-Diamond [41]、BBH [48]、ARC-Challenge [6]、HellaSwag [65] 与 TriviaQA [21]。
- 推理 (代码与数学): GSM8K [7]、MGSM [44]、Math [25]、CMath [14]、HumanEval [5] 与 MBPP [1]。
- 中文语言理解: CMMLU [26] 与 C-Eval [19]。

如表 3 所示, 块式 AttnRes 在所有基准测试中均匹配或超越基线模型。提升在多步推理任务 (如 GPQA-Diamond (+7.5) 与 Minerva Math (+3.6)) 以及代码生成任务 (如 HumanEval (+3.1)) 上尤为显著; 知识导向型基准测试 (如 MMLU (+1.1) 与 TriviaQA (+1.9)) 也展现出稳健增益。该模式与如下假设一致: 改进的深度方向信息流有利于组合型任务, 使得后续层能够有选择地检索并构建于早期表征之上。

5.3 消融研究

我们在表 2 所示的 16 头模型上开展消融研究, 以验证 AttnRes 中的关键设计选择 (表 4)。所有模型共享完全相同的超参数与计算预算。

与先前方法的对比: 我们将 AttnRes 与 PreNorm 基线模型 (损失 1.766) 以及两种推广残差连接的代表性方法进行对比。DenseFormer [36] 授予每层对所有先前输出的访问权限, 但以固定、输入无关的标量系数组合它们; 其性能未超越基线 (1.767), 凸显输入相关加权的重要性。mHC [59] 通过 m 个并行流及学习所得混合矩阵引入输入依赖性, 性能提升至 1.747。AttnRes 进一步借助 softmax 注意力实现显式的、内容相关的选取: 完整 AttnRes 达到 1.737, 块式 AttnRes 达到 1.746, 仅用每层一个查询向量便超越上述两种方法。

跨层访问: 我们比较三种粒度的跨层访问方式。完整 AttnRes 直接源于时-深对偶性 (§3), 对所有先前层应用注意力, 取得最低损失 (1.737)。降低其内存成本的一种简单方法是滑动窗口聚合 (SWA), 仅保留最近 $W=8$ 层输出及词元嵌入; 它优于基线模型 (1.764), 但仍明显落后于完整与块式 AttnRes, 表明有选择地访问远距离层比大量邻近层进行注意力更重要。

块式 AttnRes 提供更优的权衡: 当块大小 $S=4$ 时, 其达到 1.746 的性能, 同时每层内存开销保持恒定。图 6 扫描了从 $S=1$ (即完整 AttnRes) 到越来越粗粒度分组的全范围 S 值。随着 S 增大, 损失呈平缓下降趋势; $S=2, 4, 8$ 均稳定在约 1.746, 而更大块 ($S=16, 32$) 则趋近基线模型。实践中, 我们为基础设施效率 (§4) 将块数固定为约 8。未来随着硬件缓解内存容量限制, 采用更细粒度的块大小或完整 AttnRes 将成为进一步提升性能的自然路径。

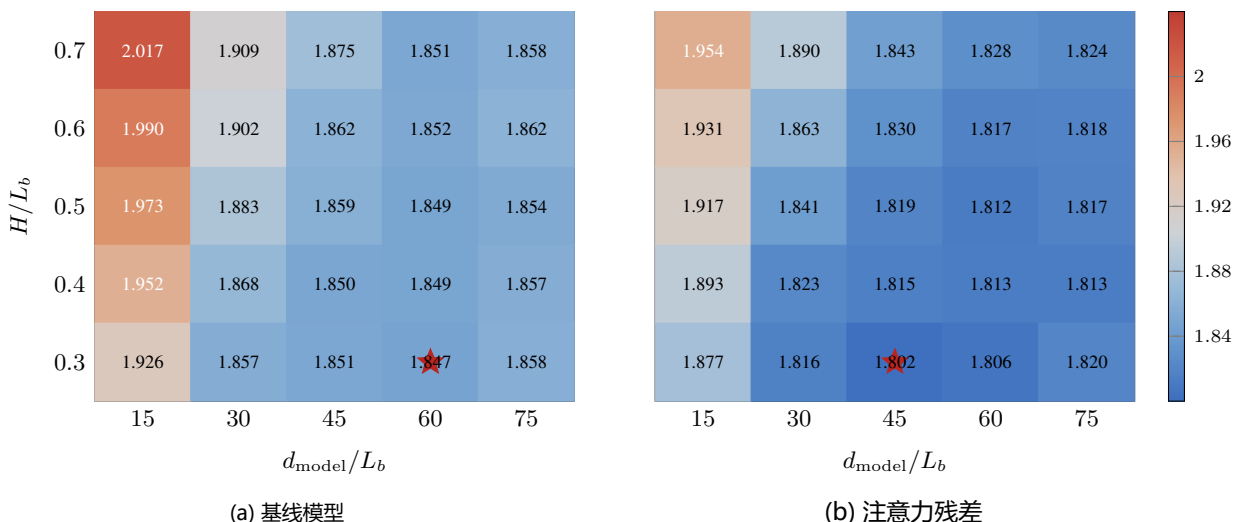


图 7: 在固定计算量 ($\approx 6.5 \times 10^{19}$ FLOPs, $\approx 2.3 \times 10^8$ 活跃参数) 下的架构扫描。每个单元格报告对应 (d_{model}/L_b , H/L_b) 配置的验证损失, 其中 $L_b = L/2$ 表示 Transformer 块数; 星号标记最优值。

组件设计: 我们进一步对注意力机制的各个组件进行消融分析:

- 输入相关查询: 一种自然延伸是使查询输入相关, 即由当前隐藏状态投影得到。该方式进一步将损失降至 1.731, 但每层引入一个 $d \times d$ 投影, 且解码时需顺序内存访问, 因此我们默认采用学习所得的查询向量。
- 输入无关混合: 我们移除了查询 (query) 与键 (key), 代之以可学习的、输入无关的标量来加权先前各层, 此操作损害性能 (1.749 对比 1.737)。
- softmax 与 sigmoid: 以 sigmoid 替代 softmax 会降低性能 (1.741)。我们认为这是由于 softmax 具备竞争性归一化能力, 能迫使模型在多个源之间做出更锐利的选择。
- 多头注意力: 我们在块式 AttnRes 上测试按头聚合深度 ($H=16$), 允许不同通道组关注不同源层。该操作损害性能 (1.752 对比 1.746), 表明最优的深度混合在通道间基本均匀: 当某层输出相关时, 其整体输出均相关。
- 键 (key) 上的 RMSNorm: 移除 RMSNorm 会降低完整 AttnRes (1.743) 与块式 AttnRes (1.750) 的性能。对完整 AttnRes 而言, 它可防止输出天然较大的单层在 softmax 中占据主导地位。这一点对块式 AttnRes 更为关键, 因为块级表征在更多层上累积, 易产生幅度差异; RMSNorm 可防止此类差异偏差注意力权重。

5.4 分析

5.4.1 最优架构

为理解 AttnRes 如何重塑最优架构缩放规律, 我们在固定计算量与参数量预算下开展受控的容量再分配研究。核心问题是: AttnRes 是否改变了深度-宽度-注意力之间的权衡偏好? 尤其鉴于其在深度维度上可能具备的优势, 它是否比传统 Transformer 设计启发式更倾向采用更深的模型? 为隔离与深度直接关联的结构因素, 我们依据内部实证观察结果固定每专家 MLP 的扩展比 ($d_{\text{ff}}/d_{\text{model}} \approx 0.45$)。我们进一步固定总训练计算量 (FLOPs $\approx 6.5 \times 10^{19}$) 与活跃参数量 ($\approx 2.3 \times 10^8$), 确保任何性能变化纯粹源于架构层面的再分配, 而非整体容量差异。在此受限预算下, 我们在 $d_{\text{model}}/L_b \in \{15, 30, 45, 60, 75\}$ 与 $H/L_b \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$ 构成的 5×5 网格上枚举了 25 种配置, 其中 $L_b = L/2$ 表示 Transformer 块数, H 表示注意力头数。结果见图 7。

两张热力图均呈现相同模式: 损失随 d_{model}/L_b 增大而减小、随 H/L_b 减小而减小; 两种方法的最优值均出现在 $H/L_b \approx 0.3$ 处。尽管趋势一致, AttnRes 在全部 25 种配置中损失均低于基线模型, 降幅为 0.019–0.063。最显著差异在于最优值位置: 基线模型在 $d_{\text{model}}/L_b \approx 60$ (1.847) 处取得最低损失, 而 AttnRes 将其移至 $d_{\text{model}}/L_b \approx 45$ (1.802)。在固定

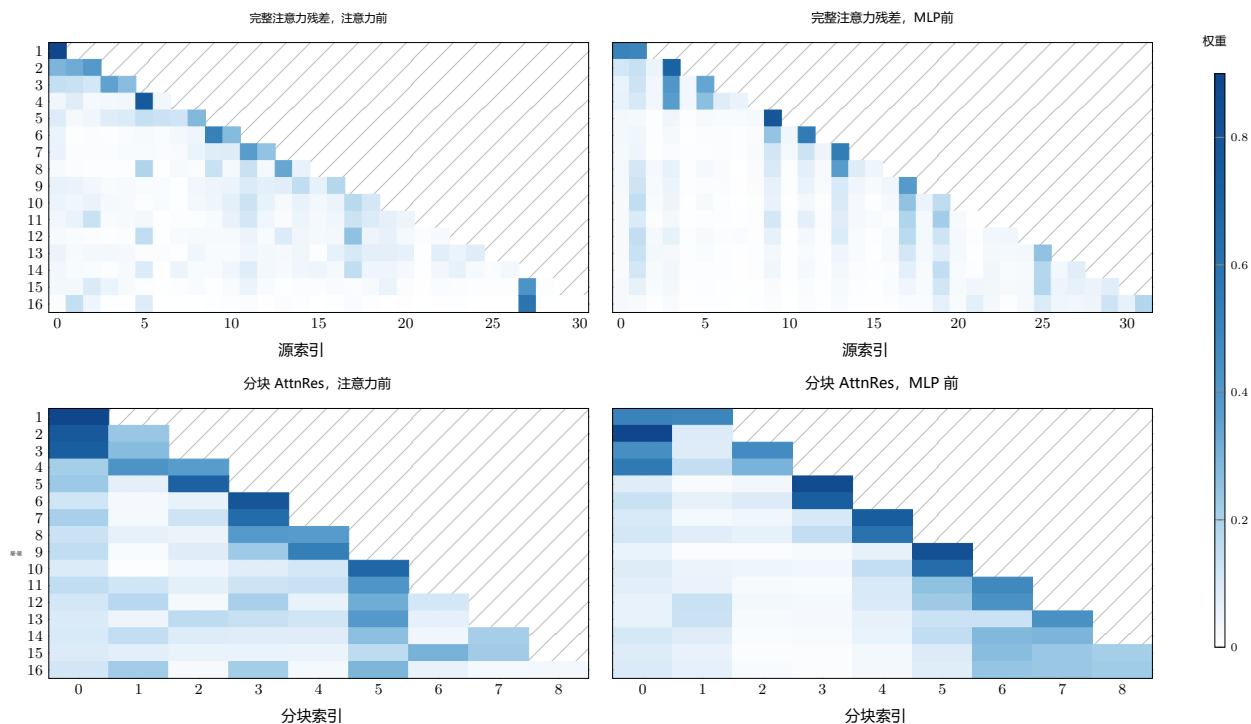


图 8: 16 头模型在完整 (上) 与分块 (下) 注意力残差连接下的逐层注意力权重分布 (按 token 平均)。该模型含 16 个注意力层与 16 个 MLP 层。每行展示第 l 个注意力 (左) 或 MLP (右) 层如何在先前源上分配权重。对角线占优表明局部性仍是主要信息通路, 而源 0 (嵌入) 上持续存在的权重以及偶发的非对角线集中现象揭示了所学习的跳跃连接。分块注意力 ($N = 8$) 以更锐利、更明确的权重分布恢复了核心结构。

参数预算相同时, 较低的 d_{model}/L_b 对应更深、更窄的网络, 表明 AttnRes 能更有效地利用额外深度。需注意, 这种对深度的偏好并不直接转化为部署建议, 因为更深模型通常因顺序计算而带来更高推理延迟 [39]。该超参扫描本质上是一种诊断手段, 用以揭示 AttnRes 效益最显著之处; 此深度偏好可在架构选型时与推理开销一并权衡。

5.4.2 分析所学习的 AttnRes 模式

我们在图 8 中可视化了 16 头模型 (来自表 2) 在完整和分块 ($N=8$) AttnRes 下所学习的权重 $\alpha_{i \rightarrow l}$ 。热图分别展示了第 l 个注意力或 MLP 层 (行) 如何在先前源 (列) 上分配其注意力权重, 并将注意力前与 MLP 前层分开呈现。我们重点指出三点观察结果:

- 保留局部性: 每层主要关注其紧邻前驱层, 但亦出现选择性的非对角线集中现象 (例如第 4 层关注早期源, 分块设置下第 15–16 层回溯更早层), 表明模型习得了标准残差路径之外的跳跃连接。
- 层特化性: 嵌入 h_1 在整个过程中始终维持非平凡权重, 尤其在注意力前各层中尤为明显。MLP 前输入对近期表征表现出更尖锐的对角线依赖, 而注意力前输入则保持更宽泛的感受野, 这与注意力机制跨层路由信息、MLP 局部运算的特性一致。
- Block AttnRes 保留结构特性。对角线占优性、嵌入持久性及层特化性均从完整版迁移至分块版, 表明分块压缩起到了隐式正则化作用, 同时保留了关键信息通路。

表 5: 残差更新机制对比。权重: 混合系数是架构固定、学习后静态 (训练完成后即固定) 还是输入依赖 (动态)。源: 第 l 层可访问的早期表征。为清晰起见, 多数公式中省略了归一化操作。

方法	更新规则	权重	源
单状态循环: 第 l 层仅接收 h_{l-1}			
残差连接 [12]	$h_l = h_{l-1} + f_{l-1}(h_{l-1})$	固定	h_{l-1}
ReZero [2]	$h_l = h_{l-1} + \alpha_l \cdot f_{l-1}(h_{l-1})$	静态	h_{l-1}
LayerScale [50]	$h_l = h_{l-1} + \text{diag}(\lambda_l) \cdot f_{l-1}(h_{l-1})$	静态	h_{l-1}
Highway [45]	$h_l = (1 - g_l) \odot h_{l-1} + g_l \odot f_{l-1}(h_{l-1})$	动态	h_{l-1}
DeepNorm [54]	$h_l = \text{Norm}(\alpha h_{l-1} + f_{l-1}(h_{l-1}))$	固定	h_{l-1}
KEEL [4]	$h_l = \text{Norm}(\alpha h_{l-1} + f_{l-1}(\text{Norm}(h_{l-1})))$	固定	h_{l-1}
多状态循环: 第 l 层接收 m 个数据流			
SiameseNorm [27]	$h_l^1 = \text{Norm}(h_{l-1}^1 + y_{l-1}); h_l^2 = h_{l-1}^2 + y_{l-1}$	固定	双流
HC/mHC [72, 59]	$H_l = H_{l-1} A_l + f_{l-1}(H_{l-1} \alpha_{l-1}) \beta_{l-1}^\top$	动态	m 个流
DDL [67]	$H_l = (I - \beta_l k_l k_l^\top) H_{l-1} + \beta_l k_l v_l^\top$	动态	dv 个流
跨层访问: 第 l 层可访问各个早期层的独立输出			
DenseNet [17]	$h_l = \text{ConvPool}([h; f(h); \dots; f(h)])$	静态	$[h, \dots, h]$
DenseFormer [36]	$h_l = \alpha \leftarrow l \cdot h + \sum_{i=1}^l \alpha_{i \rightarrow l} f_i(h_i)$	静态	$[h, \dots, h]$
MRLA [10] ¹	$h_l = \sum_{i=1}^l \phi(\text{ConvPool}(f_{l-1}(h_{l-1}))) \tau_d \text{ConvPool}(f_i(h_i)) \odot \text{Conv}(f_i(h_i))$	动态	$[h, \dots, h]$
$\text{Full2 } h_l \propto \sum_{i=0}^l \phi(w_l, k_i) v_i$ 动态 $[h, \dots, h]$ AttnRes (本文方法) $\text{Block3 } h_l \propto \sum_{i=1}^l \phi(w_l, k_i) v_i + \phi(w_l, k_l) v_l$			
		动态	$[b, \dots, b, b_l]$

1 ConvPool: 池化操作后接卷积 (通道投影)。

2 $\phi(q, k) = \exp(q \text{TRMSNorm}(k))$; $k_i = v_i$; $v = h$, $v_i \geq 1 = f_i(h_i)$ 。softmax 在所有源上联合归一化。

3 同 Full 方法, ϕ 和归一化方式一致; $v_i = b_i$, $v_{jn} = b_{jn}$ 。

6 讨论

6.1 序列-深度对偶性

残差连接通过固定循环形式 $h_l = h_{l-1} + f_{l-1}(h_{l-1})$ 在深度方向上传播信息, 其作用方式类似于 RNN 在时间维度上传播信息。测试时训练 (Test-Time Training, TTT) [46] 将这一类比的序列侧进行了形式化 (参见 Fast Weight Programmers [43, 32]), 将每个循环步骤建模为针对自监督损失的梯度下降过程:

$$W_t = W_{t-1} - \eta \nabla \ell(W_{t-1}; x_t) \quad (9)$$

其中慢速网络对 ℓ 进行参数化, 状态 W 每处理一个 token 更新一次。当 f 为线性函数时, 该式退化为标准的线性注意力 $S_t = S_{t-1} + k_t v_t^\top$ 。标准残差在深度方向上也呈现相同的加性形式: h_l 作为状态, 每一层 f_l 相当于一次梯度步。

如文献 [4] 所指出, 这种序列-深度对偶性亦适用于更丰富的变体 (见表 5)。序列侧的数据依赖型门控机制 [47, 63] 对应于深度侧的 Highway 网络 [45]; delta 规则 [42, 62, 69] 对应于 DDL [67]; 而 MRLA [10] 则与 GLAs [63] 的门控线性注意力相对应。这些方法均在保持循环范式前提下对循环更新进行了精细化改进。AttnRes 更进一步, 以直接的跨层注意力取代了深度方向上的循环结构, 正如 Transformer 以自注意力取代了时间维度上的循环结构。鉴于当前架构中的层数仍处于 softmax 注意力的实际可行范围内, 我们采用标准的深度方向注意力。将更具表达力且内存效率更高 (例如具有线性复杂度) 的替代方案纳入其中, 是未来工作的自然方向。

6.2 残差连接作为结构化矩阵

上述讨论的各类残差变体均可视为对前序层输出的加权聚合。我们使用一个深度混合矩阵 $M \in \mathbb{R}^{L \times L}$ 对此进行形式化建模, 其中 $M_{i \rightarrow l}$ 表示第 l 层赋予第 i 层输出的权重。这些变体的差异体现在权重的生成方式 (固定、可学习或依赖于输入) 以及矩阵 M 是否被约束为低秩或允许为稠密矩阵。M 的半可分秩 (semiseparable rank) [8] 为此类方法的比较提供了一种统一视角。具体而言, 第 l 层的输入为 $h_l = \sum_{i=1}^l$

过程如下所示: $v = h$ (嵌入表示), 而当 $i \geq 1$ 时, $v_i = f_i(h_i)$ 。图 9 展示了代表性方法对应的 M ; 其推导

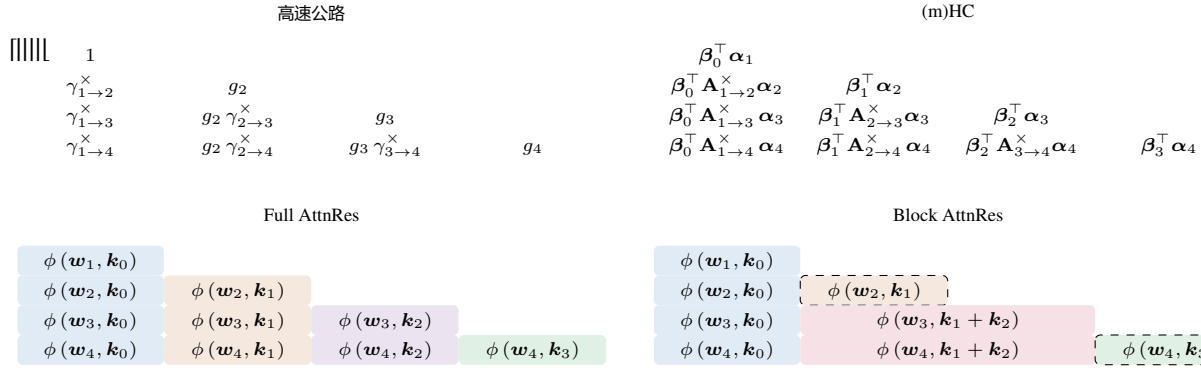


图 9：四种残差变体的深度混合矩阵 M ($L=4$ ；Block AttnRes 使用块大小 $S=2$)。为清晰起见，Highway 采用标量门控展示。AttnRes 各面板显示未归一化的 ϕ 分数；背景色将具有相同源（Full AttnRes）或相同源块（Block AttnRes）的条目分组。

- 标准残差 [12]: $h_l = h_{l-1} + f_{l-1}(h_{l-1})$ 。展开后得 $h_l = P_l - 1$ $i=0 \forall i$ ，因此对所有 $i < l$, $M_{i \rightarrow l} = 1$ ，且 M 是一个全 1 的下三角矩阵：

$$\begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_L \end{bmatrix} = \begin{bmatrix} 1 & & & \\ 1 & 1 & & \\ \vdots & \vdots & \ddots & \\ 1 & 1 & \cdots & 1 \end{bmatrix} \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{L-1} \end{bmatrix}$$

- Highway [45]: $h_l = (1 - g_l) h_{l-1} + g_l f_{l-1}(h_{l-1})$ (此处为清晰起见使用标量门控书写；逐元素扩展是直接的)。定义携带乘积 $\gamma \times$ $i \rightarrow l := Q_l j = i+1(1 - g_j)$ ，权重为 $M_{0 \rightarrow l} = \gamma \times$ $1 \rightarrow l$ 用于嵌入，且当 $i \geq 1$ 时， $M_{i \rightarrow l} = g_{i+1} \gamma \times i+1 \rightarrow l$ 。由于累积乘积经由标量门控因子分解， M 是 1-半可分矩阵 [8]，其秩与标准残差相同，但权重依赖于输入。权重按构造方式总和为 1，这使得 Highway 成为一种无需 softmax 的深度方向上的 stick-breaking 注意力 [49] 实例。
- (m)HC [72, 59] 维持 m 条并行流 $H_l \in \mathbb{R}^d \times m$ ，通过以下方式更新

$$H = HA + f(H\alpha) \beta^T,$$

其中 $A \in m \times m$ 为可学习转移矩阵， $\alpha \in m$ 将各流混合为 f 的单一输入， $\beta \in m$ 将输出重新分配至各流。展开该递归可得有效权重.....

$$M_{i \rightarrow} = \beta_i^T A \times i \rightarrow \alpha, \quad (10)$$

其中 $A_{i \rightarrow j} := Q_{ji} A$ 。 $m \times m$ 转移矩阵使 M 成为 m -半可分矩阵 [8]。mHC [59, 64] 进一步约束..... 每个 A 均被约束为双随机矩阵，以稳定跨深度的累积乘积。

- 完整版 AttnRes 通过 $\phi(w, k_i) = \exp w^T \dots$ 计算 $M_{i \rightarrow} = \alpha (i \rightarrow \text{RMSNorm}(k_i))$ (含归一化)，其中..... $k_i = v_i$ 为输入依赖的层输出，从而得到稠密、秩为 L 的 M 。
- 分块版 AttnRes 将各层划分为 N 个块 $B_1, \dots, B_N$ 。对于已完成的前序块 B 中的源层 i ，它们共享块级键/值 b ，故对每个 $i \in B$ ，均有 $M_{i \rightarrow} = \alpha \rightarrow$ 。在当前块内部，每层还额外对不断演化的部分和 b_i^n 执行注意力操作，从而在块内每个位置引入一个额外的独特源。因此， M 的有效秩介于 N 与 $N + S$ 之间 (S 为块大小)，在标准残差 ($N=1$) 与完整版 AttnRes ($N=L$) 之间形成插值。

实用性。结构化矩阵视角具有双重作用：其一，它能提供仅从递归形式本身无法获得的解析性洞见。例如，AttnRes 中输入依赖的 M 揭示了深度方向的注意力汇点 (第 5.4.2 节)，即某些层无论输入如何均持续吸引高权重，这与序列方向注意力中的同类现象 [57] 相呼应；其二，它通过揭示核函数 ϕ 的哪些性质真正关键，指导新设计的开发。例如，当 ϕ 可分解为 $\phi(q, k) = \varphi(q)^T \varphi(k)$ (其中 φ 为某特征映射) [23] 时，深度方向注意力便退化成为一种递归——这恰好构成了前述 MRLA-GLA 与 DDL-DeltaNet 对应关系的基础。

既有残差结构作为深度方向线性注意力。结构化矩阵视角进一步借助序列-深度对偶性揭示：现有残差变体实质上均为深度轴上线性注意力的特例。例如，展开后的 (m)HC 权重 $M_i \rightarrow \beta_i^T A \times$

$\rightarrow \alpha$ (式 10) 在.....中具有自然的注意力解释

其中 α 扮演第 l 层发出的查询角色， β_i 作为键，概括第 i 层的贡献，而累积转移 $A \times_i \rightarrow$ 则充当一种深度相关的位移算子 [69]，调控跨越中间各层的查询-键交互。值得注意的是， m 条并行流对应于沿深度轴的状态扩展 [40, 29]，将循环状态维度由 d 扩展至 $d \times m$ ，从而提升矩阵 M 的半可分秩。[58] 表明，即使将 $A \times_i \rightarrow$ 替换为单位矩阵，仍可获得具竞争力的性能，凸显了状态扩展的关键作用。由此视角观之，(m)HC 等方法实质上是以矩阵值状态实现的深度方向线性注意力，而 AttnRes 则实现了深度方向的 softmax 注意力。

7 相关工作

归一化、缩放与深度稳定性。标准残差更新公式 $h = h + f(h)$ [12] 在归一化位置与梯度传播之间存在根本张力。PostNorm [52] 可维持有界幅值，但会扭曲梯度，因为残差路径上重复归一化会在深度增加时加剧梯度消失现象 [60]；PreNorm [34, 60] 恢复了清晰的恒等路径，却引入了无界幅值增长：由于 $\|h\|$ 以 $O(L)$ 增长，每层的相对贡献随之衰减，迫使深层产生越来越大的输出，从而限制了有效深度 [27]。后续工作通过缩放残差路径 [54]、混合归一化 [73]、增强型跳跃连接 [4] 或可学习逐元素门控 [45] (见表 5) 调和了这两类需求。AttnRes 则绕开了这一张力，以对前序层输出的选择性聚合替代加法递归，从而避免了 PreNorm 的累积幅值增长与 PostNorm 的重复尺度收缩。

多状态递归。上述所有单状态方法仅将第 l 层建模为对 h 的条件依赖，无法从中选择性地提取各前序层的独立贡献。若干方法通过将递归扩展至多个并行流来应对这一问题：Hyper-Connections [72] 及其稳定变体 mHC [59] 维持 m 条流，并配备可学习混合矩阵；DDL [67] 维持一个通过增量擦写机制更新的矩阵状态；SiameseNorm [27] 则维持两条参数共享的流——一条 PreNorm 流与一条 PostNorm 流——以保留恒等梯度与有界表征。尽管这些方法缓解了信息压缩问题，但仍仅对紧邻前驱状态建模；AttnRes 与此正交，它提供对各前序层输出的选择性访问，同时与任意归一化或门控方案兼容。我们在第 6.2 节中讨论其与 Hyper-Connections 的形式关联。

跨层连接性。另一条研究路径通过赋予每层对各个前序层输出的直接访问权限，绕过单一状态瓶颈。最简单的方法采用静态权重：DenseNet [17] 拼接所有前序特征图；ELMo [38] 使用可学习标量权重对 layers 表征进行 softmax 加权求和；DenseFormer [36] 与 ANCRE [68] 则在训练后固定各层可学习标量系数。对于输入依赖的聚合方式，MUDDFormer [56] 通过四个解耦流上的小型 MLP 生成位置依赖权重；MRLA [10] 对所有前序层应用逐元素 sigmoid 门控，但其可分离的查询-键乘积更接近线性注意力而非基于 softmax 的检索。其他方法则以更定向的设计换取全跨层访问的简化：值残差学习 (Value Residual Learning) [71] 仅访问单个前序层；LAuReL [30] 通过前 k 层激活的低秩投影增强残差；Dreamer [24] 将序列注意力与深度注意力及稀疏专家相结合。AttnRes 结合了 softmax 归一化、输入依赖的权重与通过每层单个 d 维伪查询实现的对所有前序层的选择性访问，并引入分块结构，将计算成本从 $O(L^2)$ 降至 $O(LN)$ 。基于缓存的流水线通信与两阶段计算策略 (第 4 节) 使分块版 AttnRes 在大规模场景下切实可行，且开销可忽略不计。

结论

受序列与深度之间对偶性的启发，我们提出 AttnRes，它以可学习的、输入依赖的逐层注意力机制取代固定且均匀的残差累加方式。我们通过消融实验与缩放律实验验证该方法，结果表明其增益在不同规模下均保持稳定。由于完整版 AttnRes (Full AttnRes) 需在每一层访问所有前序层输出，跨层聚合的内存开销随 $O(Ld)$ 增长，这对当前硬件上的大规模模型而言是难以承受的。因此，我们引入分块版 AttnRes (Block AttnRes)，将各层划分为 N 个块，并在块级表征上执行注意力操作。实证表明，采用约 8 个块即可恢复完整版 AttnRes 的大部分增益；而更细粒度的分块则随着未来硬件约束的放宽，仍是一条值得探索的方向。结合跨阶段缓存与两阶段计算策略，分块版 AttnRes 在大规模场景下具备实用性，仅带来微小的训练开销与极低的推理开销。

参考文献

- [1] Jacob Austin 等人。《利用大语言模型进行程序合成》。2021 年。arXiv: 2108.07732 [cs.PL]。URL: <https://arxiv.org/abs/2108.07732>。
- [2] Thomas Bachlechner 等人。《ReZero 是你所需要的全部：在大深度下实现快速收敛》。2020 年。arXiv: 2003.04887 [cs.LG]。URL: <https://arxiv.org/abs/2003.04887>。
- [3] Dzmitry Bahdanau, Kyunghyun Cho 与 Yoshua Bengio。《通过联合学习对齐与翻译实现神经机器翻译》。2016 年。arXiv: 1409.0473 [cs.CL]。URL: <https://arxiv.org/abs/1409.0473>。
- [4] Chen Chen 与 Lai Wei。《Post-LayerNorm 回归：稳定、富有表现力且深度充足》。2026 年。arXiv: 2601.19895 [cs.LG]。URL: <https://arxiv.org/abs/2601.19895>。
- [5] Mark Chen 等人。《评测在代码上训练的大语言模型》。2021 年。arXiv: 2107.03374 [cs.LG]。URL: <https://arxiv.org/abs/2107.03374>。
- [6] Peter Clark 等人。《你以为已解决问答任务？试试 ARC——AI2 推理挑战赛》。载于：arXiv:1803.05457v1 (2018 年)。
- [7] Karl Cobbe 等人。《训练验证器以解决数学应用题》。2021 年。arXiv: 2110.14168 [cs.LG]。URL: <https://arxiv.org/abs/2110.14168>。
- [8] Tri Dao 与 Albert Gu。《Transformer 即 SSM：通过结构化状态空间对偶性构建广义模型与高效算法》。载于：CoRR abs/2405.21060 (2024 年)。DOI: 10.48550/ARXIV.2405.21060。arXiv: 2405.21060。URL: <https://doi.org/10.48550/arXiv.2405.21060>。
- [9] DeepSeek-AI 等人。《DeepSeek-V3 技术报告》。2025 年。arXiv: 2412.19437 [cs.CL]。URL: <https://arxiv.org/abs/2412.19437>。
- [10] Yanwen Fang 等人。《通过层注意力实现跨层回溯式检索》。2023 年。arXiv: 2302.03985 [cs.CV]。URL: <https://arxiv.org/abs/2302.03985>。
- [11] Andrey Gromov 等人。《深层的不合理无效性》。2025 年。arXiv: 2403.17887 [cs.CL]。URL: <https://arxiv.org/abs/2403.17887>。
- [12] Kaiming He 等人。《深度残差学习用于图像识别》。2015 年。arXiv: 1512.03385 [cs.CV]。URL: <https://arxiv.org/abs/1512.03385>。
- [13] Dan Hendrycks 等人。《评测大规模多任务语言理解能力》。2021 年。arXiv: 2009.03300 [cs.CV]。URL: <https://arxiv.org/abs/2009.03300>。
- [14] Dan Hendrycks 等人。《使用 MATH 数据集评测数学问题求解能力》。2021 年。arXiv: 2103.03874 [cs.LG]。URL: <https://arxiv.org/abs/2103.03874>。
- [15] Jordan Hoffmann 等人。《训练计算最优的大语言模型》。2022 年。arXiv: 2203.15556 [cs.CL]。URL: <https://arxiv.org/abs/2203.15556>。
- [16] Shengding Hu 等人。《MiniCPM：通过可扩展训练策略释放小型语言模型的潜力》。2024 年。arXiv: 2404.06395 [cs.CL]。URL: <https://arxiv.org/abs/2404.06395>。
- [17] Gao Huang 等人。《密集连接卷积网络》。2018 年。arXiv: 1608.06993 [cs.CV]。URL: <https://arxiv.org/abs/1608.06993>。
- [18] Yanping Huang 等人。《GPipe：利用流水线并行高效训练巨型神经网络》。载于：《NeurIPS 进展》。2019 年。
- [19] Yuzhen Huang 等人。《C-Eval：面向基础模型的多层级、多学科中文评测套件》。载于：《NeurIPS 36 进展》(2023 年)，第 62991–63010 页。
- [20] Robert A. Jacobs 等人。《局部专家的自适应混合》。载于：《神经计算》第 3 卷第 1 期 (1991 年)，第 79–87 页。DOI: 10.1162/neco.1991.3.1.79。
- [21] Mandar Joshi 等人。《TriviaQA：一个大规模远监督阅读理解挑战数据集》。载于：arXiv 预印本 arXiv:1705.03551 (2017 年)。
- [22] Jared Kaplan 等人。《神经语言模型的缩放定律》。2020 年。arXiv: 2001.08361 [cs.LG]。URL: <https://arxiv.org/abs/2001.08361>。
- [23] 安哲洛斯·卡萨罗普洛斯等人。Transformer 即循环神经网络：采用线性注意力的快速自回归 Transformer。载于：《国际机器学习大会论文集》。哈尔·道姆三世与阿蒂·辛格主编。PMLR, 2020 年，第 5156–5165 页。URL: <https://proceedings.mlr.press/v119/katharopoulos20a.html>。
- [24] 乔纳斯·克努普等人。深度递归注意力混合：赋予潜在推理应有的关注。2026 年。arXiv: 2601.21582 [cs.AI]。URL: <https://arxiv.org/abs/2601.21582>。
- [25] 艾托尔·莱夫科维奇等人。利用语言模型求解定量推理问题。2022 年。arXiv: 2206.14858 [cs.CL]。URL: <https://arxiv.org/abs/2206.14858>。

- [26] 浩南李等人。CMMLU：衡量中文大规模多任务语言理解能力。载于：《计算语言学协会2024年会议成果集：ACL 2024》。古伦·韦·库、安德烈·马丁斯与维韦克·斯里库马尔主编。泰国曼谷：计算语言学协会，2024年8月，第11260–11285页。DOI: 10.18653/v1/2024.findings-acl.671。URL: <https://aclanthology.org/2024.findings-acl.671/>。
- [27] 天宇李等人。SiameseNorm：打破前/后归一化调和障碍。2026年。arXiv: 2602.08064 [cs.LG]。URL: <https://arxiv.org/abs/2602.08064>。
- [28] 景源刘等人。Muon可扩展用于大语言模型训练。2025年。arXiv: 2502.16982 [cs.LG]。URL: <https://arxiv.org/abs/2502.16982>。
- [29] 布莱恩·马克与杰弗里·弗拉尼根。残差矩阵Transformer：扩展残差流的规模。2025年。arXiv: 2506.22696 [cs.LG]。URL: <https://arxiv.org/abs/2506.22696>。
- [30] 高拉夫·门加尼、拉维·库马尔与桑吉夫·库马尔。LAuReL：学习增强型残差层。2025年。arXiv: 2411.07501 [cs.LG]。URL: <https://arxiv.org/abs/2411.07501>。
- [31] 马克西姆·米拉科夫与纳塔莉娅·吉梅尔舍因。Softmax在线归一化器计算。2018年。arXiv: 1805.02867 [cs.PF]。URL: <https://arxiv.org/abs/1805.02867>。
- [32] 策恩斯图伦·蒙克赫达尔等人。元学习神经记忆。载于：ArXiv abs/1907.09720 (2019年)。URL: <https://api.semanticscholar.org/CorpusID:198179407>。
- [33] 迪帕克·纳拉亚南等人。使用Megatron-LM在GPU集群上高效训练大规模语言模型。2021年。arXiv: 2104.04473 [cs.CL]。URL: <https://arxiv.org/abs/2104.04473>。
- [34] 陶安Q·阮与朱利安·萨拉萨尔。无泪Transformer：改进自注意力的归一化。载于：《国际口语翻译技术研讨会论文集》。简·尼胡斯等主编。2019年。URL: <https://aclanthology.org/2019.iwslt-1.17/>。
- [35] OpenAI等。GPT-4技术报告。2024年。arXiv: 2303.08774 [cs.CL]。URL: <https://arxiv.org/abs/2303.08774>。
- [36] 马泰奥·帕利亚尔迪尼等人。DenseFormer：通过深度加权平均增强Transformer中的信息流。2024年。arXiv: 2402.02622 [cs.CL]。URL: <https://arxiv.org/abs/2402.02622>。
- [37] 博文彭等人。Yarn：大型语言模型的有效上下文窗口扩展。载于：arXiv预印本 arXiv:2309.00071 (2023年)。
- [38] 马修·E·彼得斯等人。深度上下文文化词表示。载于：《北美计算语言学协会会议论文集》。2018年，第2227–2237页。URL: <https://aclanthology.org/N18-1202/>。
- [39] 赖纳·波普等人。高效扩展Transformer推理。2022年。arXiv: 2211.05102 [cs.LG]。
- [40] 振泰等人。HGRN2：具备状态扩展的门控线性循环神经网络。2024年。arXiv: 2404.07904 [cs.CL]。
- [41] 大卫·莱因等人。GPQA：一项研究生水平、防谷歌检索的问答基准。载于：首届语言建模大会。2024年。
- [42] 伊马诺尔·施拉格、吉里·伊里与尤尔根·施密德胡伯。线性Transformer实为秘密的快速权重编程器。载于：《国际机器学习大会论文集》。玛琳娜·梅拉与张彤主编。PMLR，2021年，第9355–9366页。URL: <https://proceedings.mlr.press/v139/schlag21a.html>。
- [43] 尤尔根·施密德胡伯。学会控制快速权重记忆：动态循环网络的一种替代方案。载于：《神经计算》第4卷第1期 (1992年)，第131–139页。
- [44] 弗雷达·施等人。语言模型是多语种思维链推理器。2022年。arXiv: 2210.03057 [cs.CL]。URL: <https://arxiv.org/abs/2210.03057>。
- [45] 鲁佩什·库马尔·斯里瓦斯塔瓦、克劳斯·格雷夫与尤尔根·施密德胡伯。高速公路网络。2015年。arXiv: 1505.00387 [cs.LG]。URL: <https://arxiv.org/abs/1505.00387>。
- [46] 宇孙等人。学会（在测试时学习）：具有强表达隐状态的循环神经网络。载于：ArXiv abs/2407.04620 (2024年)。URL: <https://api.semanticscholar.org/CorpusID:271039606>。
- [47] 宇涛孙等人。保持性网络：大型语言模型中Transformer的继任者。2023年。arXiv: 2307.08621 [cs.CL]。
- [48] 米拉克·苏兹贡等人。挑战Big-Bench任务及其是否可被思维链解决。载于：arXiv预印本 arXiv:2210.09261 (2022年)。
- [49] 肖恩·谭等人。扩展Stick-Breaking注意力：一种高效实现与深入研究。载于：《国际学习表征会议论文集》。2025年。
- [50] 胡戈·图夫龙等人。用图像Transformer深入探索。2021年。arXiv: 2103.17239 [cs.CV]。URL: <https://arxiv.org/abs/2103.17239>。
- [51] 胡戈·图夫龙等人。LLaMA：开放且高效的基座语言模型。2023年。arXiv: 2302.13971 [cs.CL]。

- [52] 阿希什·瓦斯瓦尼等人。注意力就是你所需要的全部。载于：《神经信息处理系统进展》。I. 古永等编。Curran Associates, Inc., 2017年。URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf。
- [53] 阿希什·瓦斯瓦尼等人。注意力就是你所需要的全部。载于：《神经信息处理系统进展》。I. 古永等编。第30卷。Curran Associates, Inc., 2017年。URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf。
- [54] 宏宇王等人。DeepNet：将Transformer扩展至1000层。2022年。arXiv: 2203.00555 [cs.CL]。URL: <https://arxiv.org/abs/2203.00555>。
- [55] 宇博王等人。MMLU-Pro：一个更具鲁棒性与挑战性的多任务语言理解基准。载于：《神经信息处理系统进展》第37卷（2024年），第95266–95290页。
- [56] 达肖等人。MUDDFormer：通过多路动态稠密连接突破Transformer中的残差瓶颈。载于：《国际机器学习大会论文集》。2025年。
- [57] 广轩肖等人。具备注意力汇聚点的高效流式语言模型。载于：arXiv预印本arXiv:2309.17453（2023年）。
- [58] 天谢。你的DeepSeek mHC或许根本不需要‘m’。知乎专栏文章。2026年。URL: <https://zhuanlan.zhihu.com/p/2010852389670908320>。
- [59] 增达谢等人。mHC：流形约束的超连接。2026年。arXiv: 2512.24880 [cs.CL]。URL: <https://arxiv.org/abs/2512.24880>。
- [60] 睿斌熊等人。论Transformer架构中的层归一化。2020年。arXiv: 2002.04745 [cs.LG]。URL: <https://arxiv.org/abs/2002.04745>。
- [61] 博文杨等人。从RoPE到NoPE再回归：一种新型混合注意力策略。2025年。arXiv: 2501.18795 [cs.CL]。URL: <https://arxiv.org/abs/2501.18795>。
- [62] 松林杨、扬·考茨与阿里·哈塔米扎德。门控增量网络：利用增量规则改进Mamba2。载于：《国际学习表征会议论文集》。2025年。URL: <https://openreview.net/forum?id=r8H7xhYPwz>。
- [63] 松林杨等人。具备硬件高效训练能力的门控线性注意力Transformer。载于：《国际机器学习大会论文集》。PMLR, 2024年。
- [64] 永毅杨与建阳高。mHC-lite：你无需执行20次Sinkhorn-Knopp迭代。2026年。arXiv: 2601.05732 [cs.LG]。URL: <https://arxiv.org/abs/2601.05732>。
- [65] 罗万·泽勒斯等人。HellaSwag：机器真能续写你的句子吗？载于：《计算语言学协会第57届年会论文集》。2019年。
- [66] 彪张与里科·森里奇。均方根层归一化。载于：《神经信息处理系统进展》第32卷（2019年）。
- [67] 一凡张等人。深度增量学习。2026年。arXiv: 2601.00417 [cs.LG]。URL: <https://arxiv.org/abs/2601.00417>。
- [68] 一朗张等人。ANCRE：面向高效深度缩放的自适应神经连接重分配。2026年。arXiv: 2602.09009 [cs.LG]。URL: <https://arxiv.org/abs/2602.09009>。
- [69] 余张等人。Kimi Linear：一种富有表现力且高效的注意力架构。2025年。arXiv: 2510.26692 [cs.CL]。
- [70] 钟舒等。从关联记忆视角理解Transformer。2025。arXiv: 2505.19488 [cs.LG]。URL: <https://arxiv.org/abs/2505.19488>。
- [71] 周展超等。Value Residual Learning。载于：ACL会议论文集。主编：车万翔等。奥地利维也纳，2025，第28341–28356页。URL: <https://aclanthology.org/2025.acl-long.1375/>。
- [72] 朱德发等。Hyper-Connections。2025。arXiv: 2409.19606 [cs.LG]。URL: <https://arxiv.org/abs/2409.19606>。
- [73] 卓志健等。HybridNorm：通过混合归一化实现稳定高效的Transformer训练。2025。arXiv: 2503.04598 [cs.CL]。URL: <https://arxiv.org/abs/2503.04598>。

A 贡献说明

作者按贡献重要性排序，项目领导角色者列于最后。

陈广宇* 张宇*
苏剑林* 徐维
新 潘思源 王
尧宇 王宇成
陈冠铎 尹博弘
陈玉田 闫俊
杰 魏明

陈彦儒 门鑫
顾海清 郭晓
宇 卢浩宇 隋
林 朱金国 周
子达 何伟然
黄伟骁 徐欣
然 王玉芝 赖
国坤 杜宇伦
吴宇昕 杨志
林 周新宇

张宇 孟凡青
洪超 谢晓彤
刘少威 陆恩
哲 台云鹏

* 贡献相同

B 全注意力残差的优化推理 I/O

Full AttnRes 的朴素实现会在每一层扫描所有先前层的输出，导致内存通信量随深度线性增长。但如 §4.2 所述，伪查询 w_l 是一个与输入及隐状态均无关的可学习参数。因此，我们可通过两阶段调度策略跨层批处理块间访问，从而显著降低总 I/O，远低于朴素实现的上界。

注意，下文引入的块划分纯粹是一种推理调度机制。与 Block AttnRes 不同，它不改变模型架构，也不以块摘要替代每层源；它仅使摊销分析更加明确具体。

设定：模型共有 L 层，隐藏维度为 d ，划分为 N 个大小为 $S = L/N$ 的连续块。推理过程逐块进行：第一阶段联合计算块内全部 S 层对所有先前块的块间注意力；第二阶段按顺序遍历块内依赖关系。

第一阶段：批处理块间注意力

考虑第 n 个块及其 S 层。查询向量 $\{w_l\}_{l \in B_n}$ 在执行开始前均已知，因此只需从 HBM 中一次性读取前面 $(n-1)S$ 对键值对，并在全部 S 个查询中复用。第 n 个块的读取开销因此为

$$\text{Read}_{\text{inter}}^{(n)} = 2(n-1)Sd, \quad (11)$$

其中系数 2 表示同时读取键与值。对全部 N 个块求和，并利用 $SN = L$ ：

$$\text{Read}_{\text{inter}} = \sum_{n=1}^N 2(n-1)Sd = 2Sd \cdot \frac{N(N-1)}{2} = dL(N-1). \quad (12)$$

第一阶段同样为每层写入一个 d 维输出，因此 $\text{Write}(n)_{\text{inter}} = Sd$ （每块），且

$$\text{Write}_{\text{inter}} = Ld \quad (13)$$

总计。

第二阶段：顺序块内注意力

第一阶段覆盖当前块之前的所有源。然而，在块内，每一层均依赖于其前面的层，因此这些层必须按顺序处理。第 t 层 ($1 \leq t \leq S$) 读取 $t-1$ 对块内键值对，开销为 $2(t-1)d$ 。对一个块内所有层求和：

$$\text{Read}_{\text{intra}}^{(n)} = \sum_{t=1}^S 2(t-1)d = S(S-1)d. \quad (14)$$

第二阶段同样为每层写入一个输出，因此 $\text{Write}(n)_{\text{intra}} = Sd$ 。

每层平均摊销 I/O

对全部 N 个块求和两个阶段：

$$\text{总读取量} = dL(N-1) + N \cdot S(S-1)d, \text{ 总写入量} = 2Ld. \quad (15)$$

除以 L 并利用 $SN = L$ ：

$$\text{每层读取量} = (N-1)d + (S-1)d = (S+N-2)d, \text{ 每层写入量} = 2d, \quad (16)$$

$$\boxed{\text{每层总 I/O} = (S+N)d.} \quad (17)$$

因此，对块间读取进行批处理可将每层的 I/O 从 $O(L)$ 降低至 $O(S+N)$ 。该调度策略沿用了 Block AttnRes 的相同两阶段划分：块间注意力占据了绝大部分通信流量，而顺序计算则保持在每个块内部完成。